

Mauerstrassen Agent

Aktuelle Themen und Trends I und II

BWSE - SS 2026

Group 7

Dario-Benjamin Dzakic – 2310859034

Marco Magyar – 2410859018

Abstract—Tick the boxes for the level your implementation achieves:

- ☐ Minimal
- ☐ Advanced
- ☒ Expert

I. AUFGABENVERTEILUNG

Dario-Benjamin Dzakic hat die ersten drei Agents entworfen und implementiert: *Strategy Advisor*, *Market Analyst* und *Stock Screener*, samt der zugehörigen Tasks (`strategy_task`, `trend_task`, `screening_task`). Er hat die RAG-Wissensquelle für Sektoranalyse (`sector_analysis.md`) erstellt und den Finnhub-MCP-Server (`finnhub_server.py`) geschrieben, der sechs Finanzdaten-Tools bereitstellt (`get_company_profile`, `get_basic_financials`, `get_financials_reported`, `get_company_news`, `get_recommendation_trends`, `get_quote`).

Marco Magyar hat die restlichen zwei Agents entworfen und implementiert: *Stock Checker* und *Portfolio Builder*, samt den zugehörigen Tasks (`checker_task`, `portfolio_task`). Er hat die RAG-Wissensquellen für Anlagestrategien und Risikorahmen (`investment_strategies.md`, `risk_frameworks.md`) erstellt und die MCP-CrewAI-Integrationsschicht (`mcp_servers.py`) entwickelt, die den Finnhub-MCP-Server über `MCPServerAdapter` an das CrewAI-Tool-Ökosystem anbindet. Außerdem hat er die gesamte Crew-Orchestrierung designed, inklusive hierarchischem Prozess mit Manager-LLM und dem Google Generative AI Embedding-Provider.

Beide Teammitglieder haben an Architektur-Design, Tests und Dokumentation mitgearbeitet.

II. PROBLEMSTELLUNG

Ein Aktienportfolio von Hand zusammenzustellen ist mühsam. Man muss seine Risikobereitschaft einschätzen, Markttrends verfolgen, dutzende Ticker durchforsten, die Fundamentaldaten prüfen und dann irgendwie entscheiden, wie viel Geld wohin geht. Die meisten Leute überspringen die Hälfte dieser Schritte oder verlieren sich in Analyse-Paralyse.

Wir haben **Mauerstrassen Agent** gebaut—ein CrewAI-Multi-Agent-System, das den ganzen Prozess übernimmt. Man gibt an, wie viel Kapital man hat, wie riskant man investieren will und welchen Zeithorizont man anpeilt; zurück kommt

ein Portfolio mit konkreten Positionen, Kaufkursen und Stop-Losses.

Wir gehen davon aus, dass man bereits ein Brokerage-Konto hat, dass die Daten von Finnhub und der Serper-Websuche aktuell genug sind und dass unser Output eine *Empfehlung* ist—keine regulierte Finanzberatung.

III. ARCHITEKTUR

Das System ist eine CrewAI-Crew mit fünf Agents, gesteuert durch einen hierarchischen Prozess—im Grunde ein Manager-LLM, das Aufgaben verteilt und die Ergebnisse zusammenfügt. Drei Technologien machen das Ganze möglich:

- 1) **RAG**: Wir haben eigene Wissensdokumente (Anlagestrategien, Risikorahmen, Sektoranalyse) eingebettet, damit Agents zur Laufzeit Informationen nachschlagen können.
- 2) **MCP**: Ein eigener Finnhub-MCP-Server liefert Echtzeit-Finanzdaten über stdio an die Agents.
- 3) **Websuche**: Serper gibt Agents Zugang zu aktuellen Marktnachrichten und Trends.

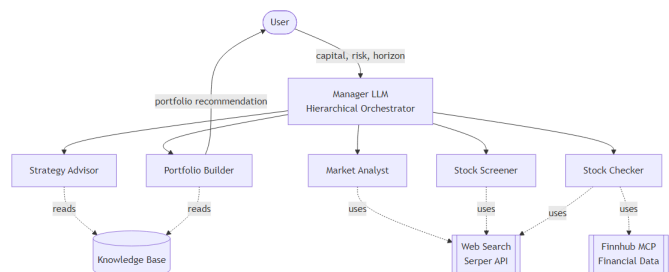


Fig. 1. High-level system architecture. The Manager LLM orchestrates five specialized agents, each connected to external tools (Knowledge Base, Web Search, Finnhub MCP).

A. Agent-Beschreibung

Wir haben fünf Agents:

- 1) **Strategy Advisor** — Spricht mit dem User, um Kapital, Risikotoleranz und Zeithorizont zu erfassen. Gleicht die Antworten mit unserer RAG-Wissensbasis ab, um Widersprüche zu erkennen (z. B. aggressives Risiko bei 3-Monats-Horizont). Gibt eine strukturierte Strategie aus.

- 2) **Market Analyst** — Durchsucht das Web nach fünf Markttrends, die zur Strategie passen, und lässt den User auswählen, welche er verfolgen will.
- 3) **Stock Screener** — Findet pro gewählttem Trend bis zu fünf Aktien, die profitieren könnten, gefiltert nach Marktkapitalisierung und Liquidität.
- 4) **Stock Checker** — Der Finnhub-gestützte Validierungsschritt. Holt Profile, Finanzkennzahlen, News und Analysten-Empfehlungen über MCP-Tools und gibt jeder Aktie ein Go/No-Go.
- 5) **Portfolio Builder** — Nimmt die überlebenden Aktien und die Strategie des Users und baut das finale Portfolio: Allokationen, Stückzahlen, Kaufkurse, Stop-Losses. Schreibt eine Markdown-Tabelle auf die Festplatte.

B. Task-Beschreibung

Jeder Agent hat einen Task. Sie sind über Kontext-Abhängigkeiten verkettet:

- 1) **Strategy Task** — User-Input sammeln, Strategie erstellen.
- 2) **Trend Task** — Fünf Trends finden, User wählen lassen. Hängt von der Strategie ab.
- 3) **Screening Task** — Aktien pro Trend finden. Hängt von Strategie + Trends ab.
- 4) **Checker Task** — Aktien über Finnhub MCP validieren. Hängt von den Screening-Ergebnissen ab.
- 5) **Portfolio Task** — Finale Allokationstabelle bauen. Hängt von Strategie + Checker-Output ab. Schreibt nach `output/portfolio_recommendation.md`.

C. Ablauf

Das Manager-LLM führt die fünf Tasks der Reihe nach aus: Strategie → Trends → Screening → Checking → Portfolio. Die ersten beiden Tasks fragen den User nach Input, ab dem Screening läuft alles vollautomatisch.

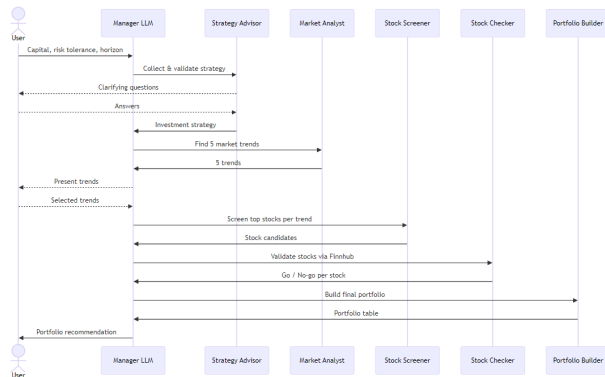


Fig. 2. Interaction flow. The Manager LLM delegates tasks sequentially: strategy formulation, trend discovery (both with human input), stock screening, validation via Finnhub, and portfolio construction.

IV. MINIMAL

Das Minimallevel verlangt mindestens vier Agents, Tool-Nutzung und MCP-Server-Integration. Wir erfüllen alle drei Punkte:

- **Mindestens 4 Agents:** Unsere Crew besteht aus fünf spezialisierten Agents (Strategy Advisor, Market Analyst, Stock Screener, Stock Checker, Portfolio Builder), definiert in `agents.yaml` mit eigenen Rollen und Goals.
- **Tool Use:** Der Market Analyst und Stock Screener nutzen das Serper-Websearch-Tool, um aktuelle Marktdaten und Trends abzurufen.
- **MCP-Server-Integration:** Der Stock Checker greift über einen Finnhub-MCP-Server auf sechs Finanzdaten-Tools zu (Unternehmensprofile, Kennzahlen, News, Kurse etc.), die über das stdio-Protokoll bereitgestellt werden.

Die fünf Tasks sind über Kontext-Abhängigkeiten verkettet (`tasks.yaml`), und das finale Ergebnis wird in ein Pydantic-Output-Schema (`PortfolioRecommendation`) gegossen. Alles ist in `crew.py` über den `@CrewBase`-Dekorator verdrahtet.

V. ADVANCED

Das Advanced-Level verlangt mehr als vier Agents und einen eigenen MCP-Server. Beides haben wir, plus zusätzliche Features:

- **Mehr als 4 Agents:** Mit fünf Agents übertreffen wir die Mindestanzahl. Jeder Agent hat eine klar abgegrenzte Rolle im Pipeline-Ablauf.
- **Eigener MCP-Server:** Wir haben einen eigenen Finnhub-MCP-Server (`finnhub_server.py`) mit FastMCP gebaut, der sechs Tools über stdio bereitstellt: `get_company_profile`, `get_basic_financials`, `get_financials_reported`, `get_company_news`, `get_recommendation_trends` und `get_quote`. Auf der CrewAI-Seite startet `MCPServerAdapter` den Server als Subprozess und konvertiert dessen Tools in CrewAI-kompatible Objekte.
- **Human-in-the-loop:** Die ersten zwei Tasks fragen den User nach Input, damit das Portfolio auch wirklich das widerspiegelt, was er will.
- **Markdown-Artefakt:** Das finale Portfolio wird als lesbares Markdown auf die Festplatte geschrieben.

VI. EXPERT

Das Expert-Level verlangt Retrieval Augmented Generation (RAG) und einen Advanced Flow. Wir haben beides implementiert.

A. RAG

Wir haben drei Wissensdokumente geschrieben und eingebettet, damit Agents zur Laufzeit Informationen nachschlagen können:

- `investment_strategies.md` — Value-, Growth-, Momentum-, Dividenden- und passive Strategien.
- `risk_frameworks.md` — Konservative / moderate / aggressive Profile mit Allokationsgrenzen und Widerspruchs-Flags.

- `sector_analysis.md` — Sieben Sektoren mit Treibern, Risiken und Rotationsmustern.

Der Strategy Advisor und der Portfolio Builder bekommen diese als Wissensquellen injiziert, damit sie fundiertes Domänenwissen haben, statt sich nur auf das zu verlassen, was das Basis-LLM weiß. Embeddings laufen über Google Generative AI.

B. Advanced Flow (Hierarchisch)

Statt eines einfachen sequentiellen Prozesses nutzen wir einen hierarchischen Prozess mit einem Manager-LLM, das die gesamte Crew koordiniert—quasi wie ein Senior-Portfoliomanager, der ein Analysten-Team leitet. Das Manager-LLM entscheidet, welcher Agent wann dran ist, fasst Zwischenergebnisse zusammen und stellt sicher, dass jeder Task den Kontext seiner Vorgänger erhält. Jeder Task deklariert explizit seine Kontext-Abhängigkeiten, damit nichts verloren geht. Das geht über einen simplen Pipeline-Ablauf hinaus, weil die Orchestrierung selbst von einem LLM übernommen wird.

VII. REFLEXION

Ein paar Sachen haben uns beim Entwickeln Probleme bereitet:

Finnhub-Antworten sind riesig. `get_financials_reported` liefert komplette Finanzberichte über mehrere Perioden. Wenn der Stock Checker das für mehrere Aktien hintereinander aufgerufen hat, ist der Kontext explodiert und das LLM hat frühere Ergebnisse vergessen. Wir sollten wohl innerhalb des MCP-Tools zusammenfassen oder kürzen.

Human-Input-Konfiguration passte nicht zusammen. Wir haben `human_input=True` auf zwei Tasks gesetzt, aber die Crew selbst nicht für menschliche Interaktion konfiguriert, was in manchen Umgebungen zu merkwürdigem Blockieren geführt hat. Lektion: Task-Level- und Crew-Level-Flags synchron halten.

Aktienanzahl schwankt. Wir haben Top 5 Aktien pro Trend verlangt, aber das LLM hat manchmal drei oder sieben zurückgegeben. Exakte Anzahlen bei Freitext-Generierung durchzusetzen ist schwierig—ein strikteres Output-Schema würde helfen.

Strukturierter Output ist unzuverlässig. Das Pydantic-Schema für das finale Portfolio wurde nicht immer eingehalten. Felder wurden übersprungen oder falsch formatiert. Ohne expliziten JSON-Mode oder Function-Calling-Erzwingung neigt der Agent dazu, einfach Freitext-Markdown zu schreiben.